Cyberscope

A TAC Security Company

Source Code Review Report

Crush Wallet

July 2026

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Source	Source Code Files

Audit Updates

Initial Audit	22nd February 2026 - 16th March 2026
Revalidated Audit	09th June 2026 – 09th June 2026
Revalidated Audit	07th July 2026 – 08th July 2026

Overview

Cyberscope has conducted a comprehensive Source Code Review for "Crush Wallet". The source code review was performed using a manual, risk-based approach, supplemented by automated analysis where appropriate. The assessment focused on security-critical components and high-risk code paths, including authentication mechanisms, sensitive data handling, external integrations, and business logic. The review began with an architectural understanding of the application, followed by systematic examination of source files to identify potential vulnerabilities, insecure API usage, hardcoded secrets, and improper error or logging practices. Findings were validated through contextual analysis to reduce false positives and assess real-world exploitability. All identified issues were documented with clear descriptions, risk ratings, and remediation recommendations based on secure coding guidelines and applicable security standards.

With the expansion of blockchain technology and related integrations, source code review is a structured security assessment technique used to identify vulnerabilities, insecure coding practices, and logic flaws directly within an application's source code. Unlike black-box testing, source code review provides full visibility into the application's implementation, enabling the detection of security weaknesses that may not be observable during runtime testing. This approach is particularly effective for identifying issues related to authentication, authorization, data storage, cryptography, input validation, and error handling. The review is conducted in alignment with industry-recognized standards and best practices to ensure the application meets established security requirements.

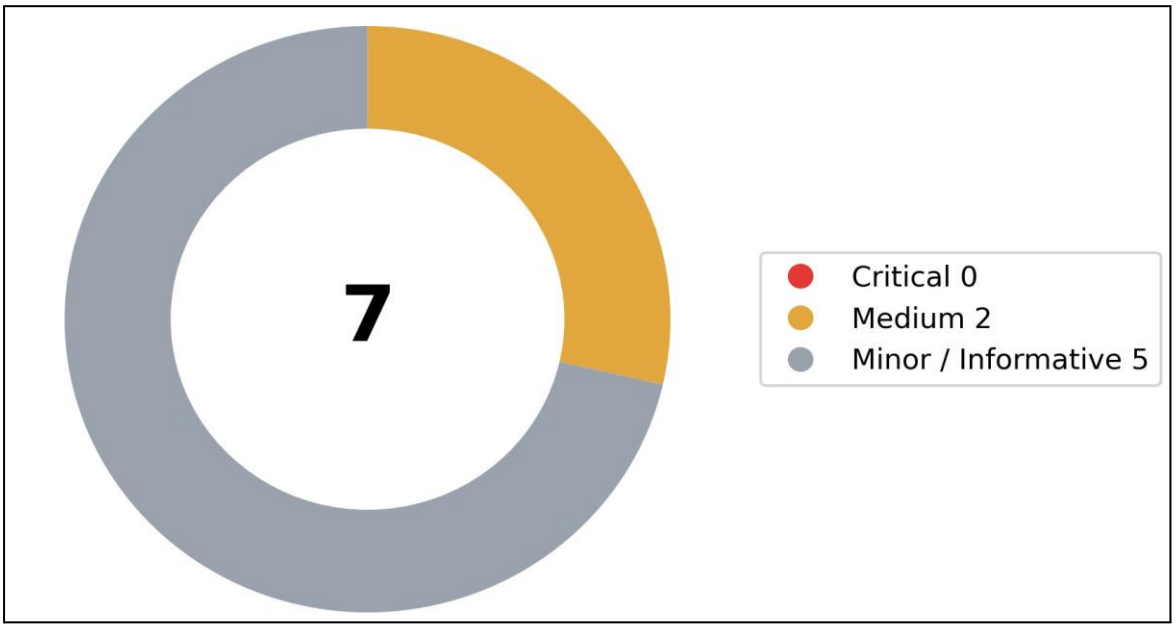
Source Code Review Scope

The scope of this assessment focused on identifying security vulnerabilities and weaknesses within the application's source code and overall implementation, with the objective of providing actionable recommendations to improve its security posture. The review examined relevant portions of the codebase, with emphasis on security-critical components, including authentication and authorization logic, sensitive data handling, cryptographic implementations, input validation and output encoding, error handling, logging practices, and interaction with external services and dependencies.

Particular attention was given to common software security risks and insecure coding patterns referenced in industry standards such as OWASP Top 10, OWASP Mobile/Web Security Guidelines, and CWE, including hardcoded secrets, improper access control enforcement, insecure deserialization, injection vulnerabilities, security misconfigurations, and inadequate protection of sensitive information.

This report aims to provide a comprehensive understanding of the application's code-level security strengths and areas for improvement, enabling informed decisions to remediate identified issues, reduce attack surface, and enhance the overall security resilience of the application.

Findings Breakdown



Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	2	0
Minor / Informative	0	0	5	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Medium	NDSWU	No Docker Secrets Were Used	Resolved
Medium	NAI	No Authentication Implemented	Resolved
Minor / Informative	BINPBD	Base Image is Not Pinned By Digest	Resolved
Minor / Informative	VOC	Vulnerable and Outdated Component	Resolved
Minor / Informative	SE	Swagger Enabled	Resolved
Minor / Informative	ACAOHWW	Access-Control-Allow-Origin Header with Wildcard.	Resolved
Minor / Informative	IME	Insecure Methods Enabled	Resolved

NDSWU - No Docker Secrets Were Used

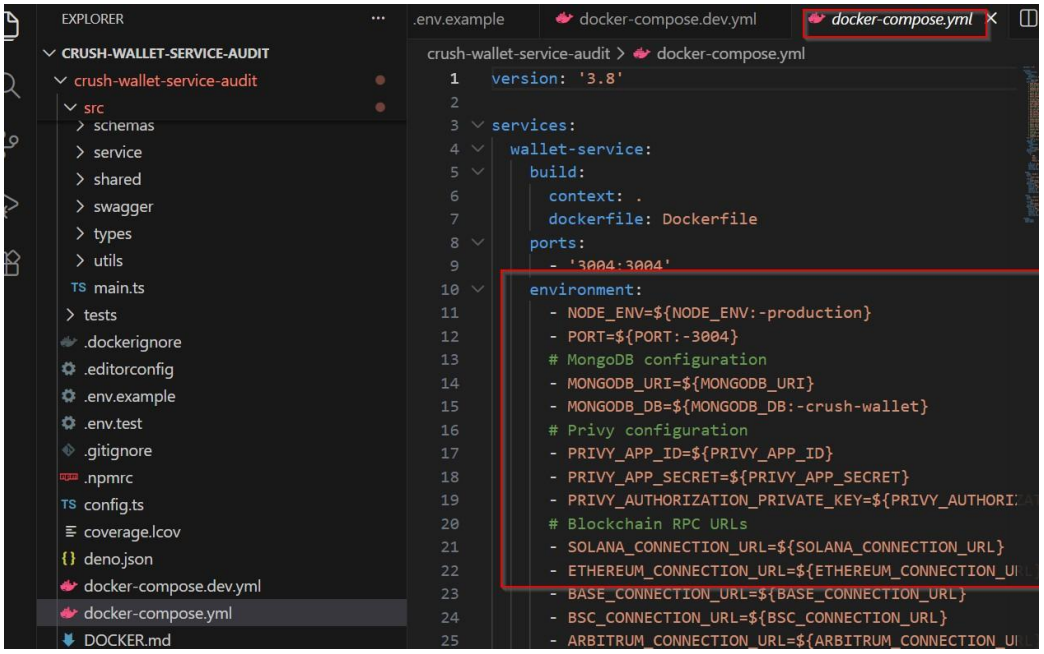
Criticality	Medium
Status	Resolved
CWE ID:	522
CVSS3 Score:	5.5

Description

The review identified that the application container configuration does not utilize Docker Secrets for managing sensitive information such as credentials, API keys, or tokens. Instead, sensitive data appears to be embedded directly within configuration files, environment variables, or container build instructions. This practice increases the exposure risk of secrets during image distribution, deployment, and runtime operations.

Storing secrets directly within container configurations or environment variables increases the likelihood of credential exposure through container image repositories, logs, backups, or compromised containers. Attackers gaining access to the container or image registry may retrieve sensitive credentials, potentially enabling unauthorized access to backend services, databases, or internal systems.

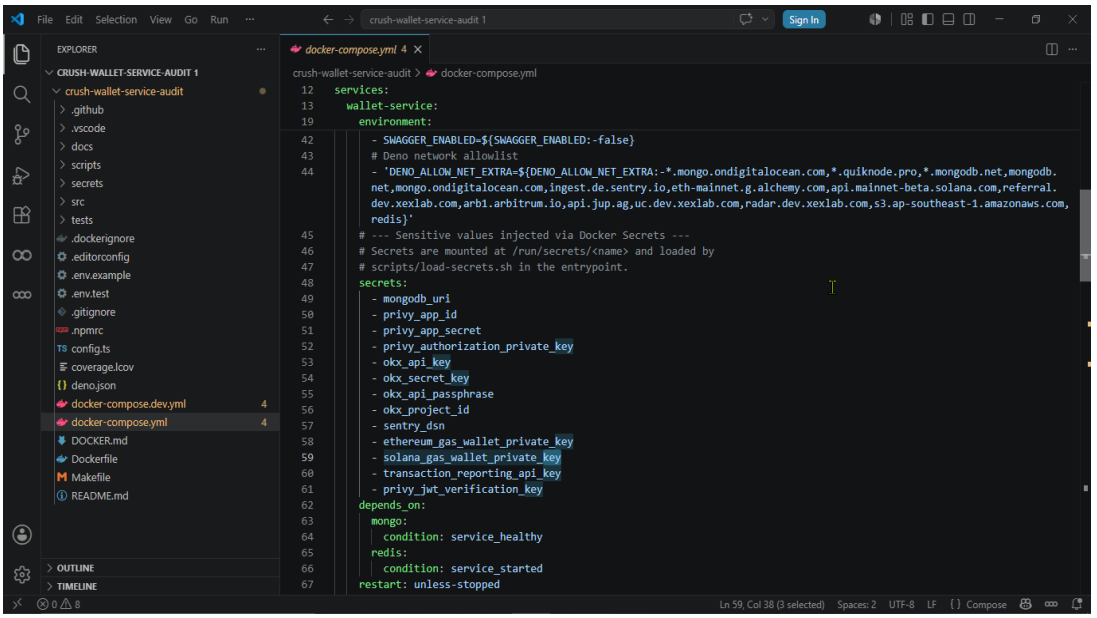
Proof of Concept: The below screenshots depict that no docker secrets used in the following docker config file.



Recommendation

It is recommended to implement Docker Secrets or a secure secrets management solution such as HashiCorp Vault, Kubernetes Secrets, or cloud-based secret managers. Sensitive values should not be embedded in images or source code. Access to secrets should follow the principle of least privilege and be dynamically injected at runtime.

Revalidated Proof of Concept: In production, the wallet service uses Docker Secrets. The docker-compose.yml file loads all sensitive values, including the database URL, API keys, private keys, Sentry DSN, and other secrets, from Docker Secrets at runtime. As a result, sensitive information is not hardcoded in the compose file.



NAI - No Authentication Implemented

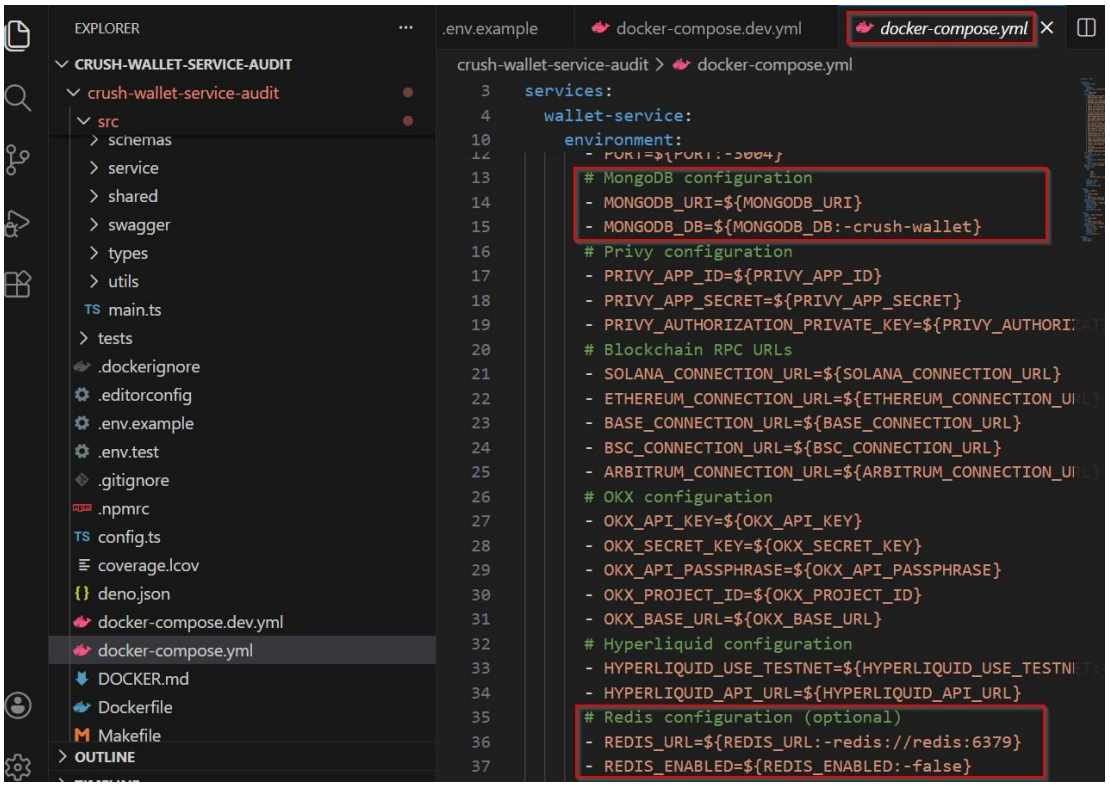
Criticality	Medium
Status	Resolved
CWE ID:	306
CVSS3 Score:	6.5

Description

The source code analysis revealed that certain application endpoints or functionalities are accessible without any authentication or identity validation mechanism. These endpoints allow users to access resources or perform operations without verifying their identity, indicating that authentication controls are either missing or improperly implemented.

The absence of authentication mechanisms can allow unauthorized users to access sensitive application features or data. Attackers may exploit such endpoints to perform unauthorized operations, extract confidential information, manipulate application behavior, or potentially escalate the attack surface leading to broader compromise of the application environment.

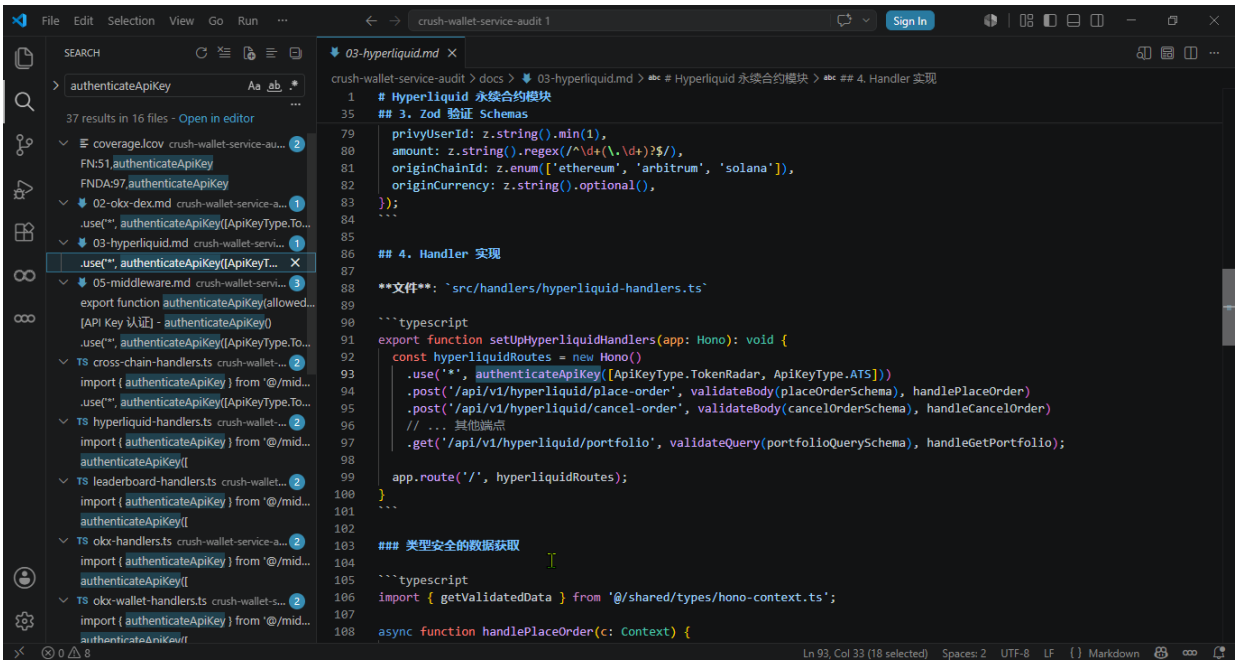
Proof of Concept: The below screenshot depicts that authentication is not implemented.



Recommendation

It is recommended to implement robust authentication mechanisms such as token-based authentication, OAuth2, or session-based authentication depending on the application architecture. Access to all critical endpoints should require identity verification. Additionally, enforce role-based or attribute-based access control to ensure only authorized users can access sensitive functionality.

Revalidated Proof of Concept: The below screenshot depicts authentication is implemented through the authenticateApiKey middleware, which is applied to the API routes, ensuring that requests are authenticated before accessing protected endpoints.



BINPBD - Base Image is Not Pinned By Digest

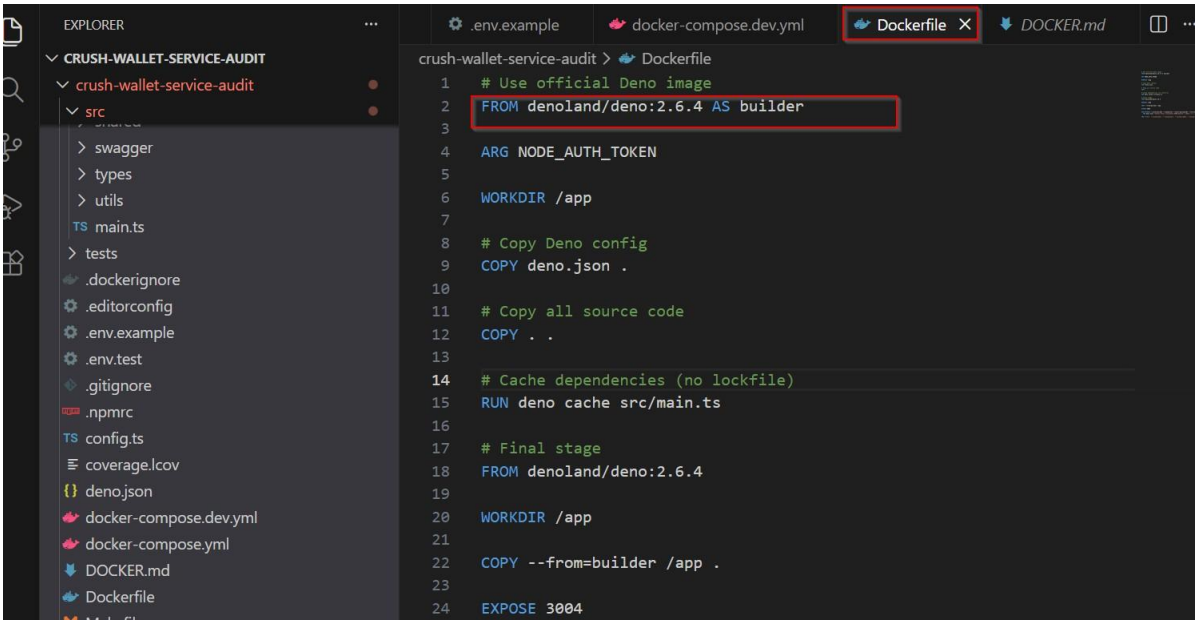
Criticality	Minor
Status	Resolved
CWE ID:	1104
CVSS3 Score:	3.7

Description

The Docker configuration uses a base image referenced only by a tag (e.g., latest or version tag) instead of a specific immutable image digest. Tags can change over time, causing builds to pull different image versions during each build process, which may introduce unintended or unverified updates into the container environment.

Using mutable image tags may result in inconsistent builds and potential introduction of malicious or vulnerable image versions. If a base image repository is compromised or updated unexpectedly, the application could unknowingly inherit insecure components, affecting the stability, integrity, and security of the deployed container.

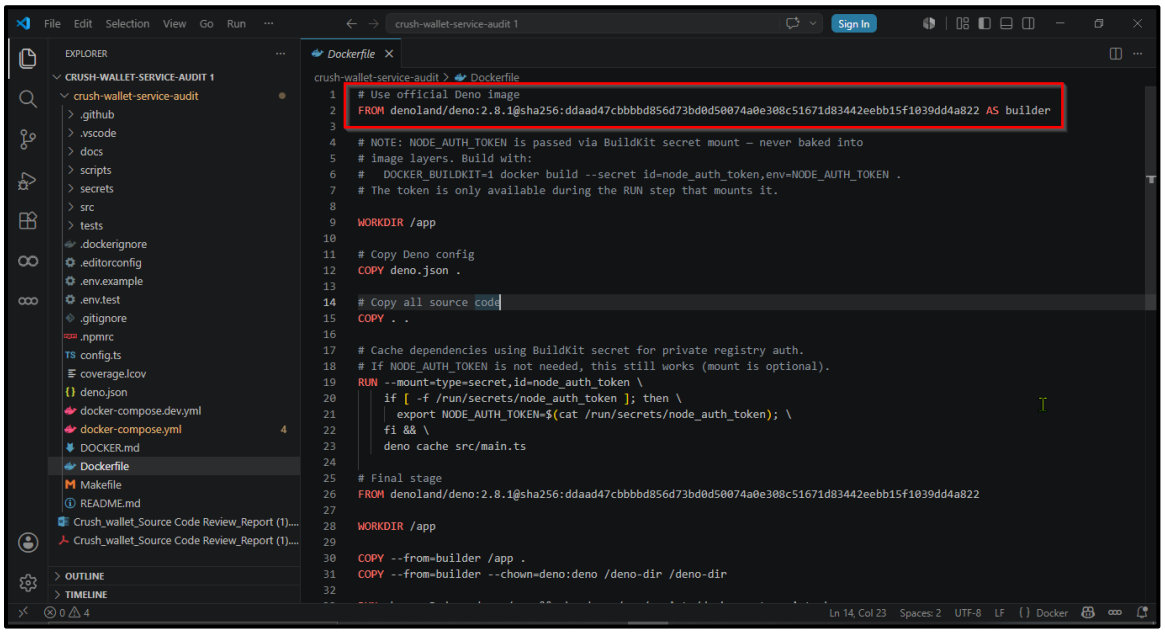
Proof of Concept: The below screenshot depicts that Base Image is Not Pinned By Digest.



Recommendation

It is recommended to pin Docker base images using a specific image digest (SHA256). This ensures consistent and reproducible builds by always referencing the exact same base image. Additionally, implement automated container image vulnerability scanning and regularly update base images through controlled and verified processes.

Revalidated Proof of Concept: The below screenshot depicts that Base Image is Pinned By Digest.



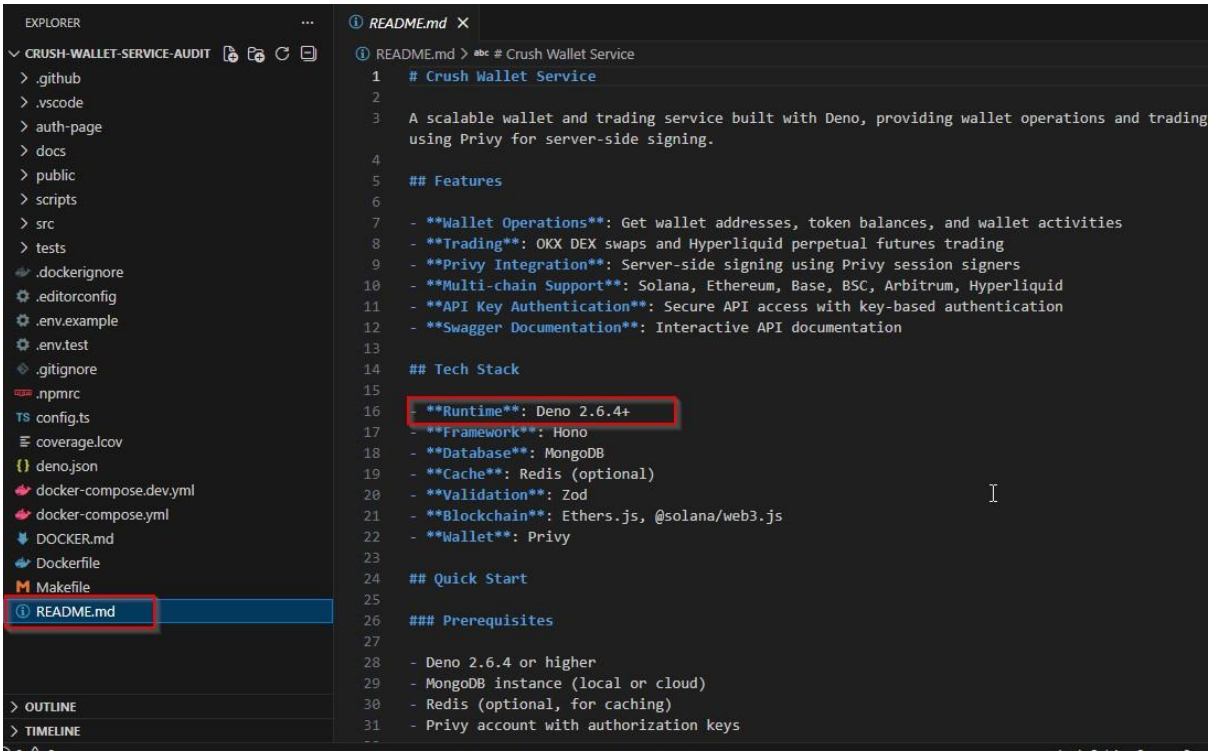
VOC - Vulnerable and Outdated Component

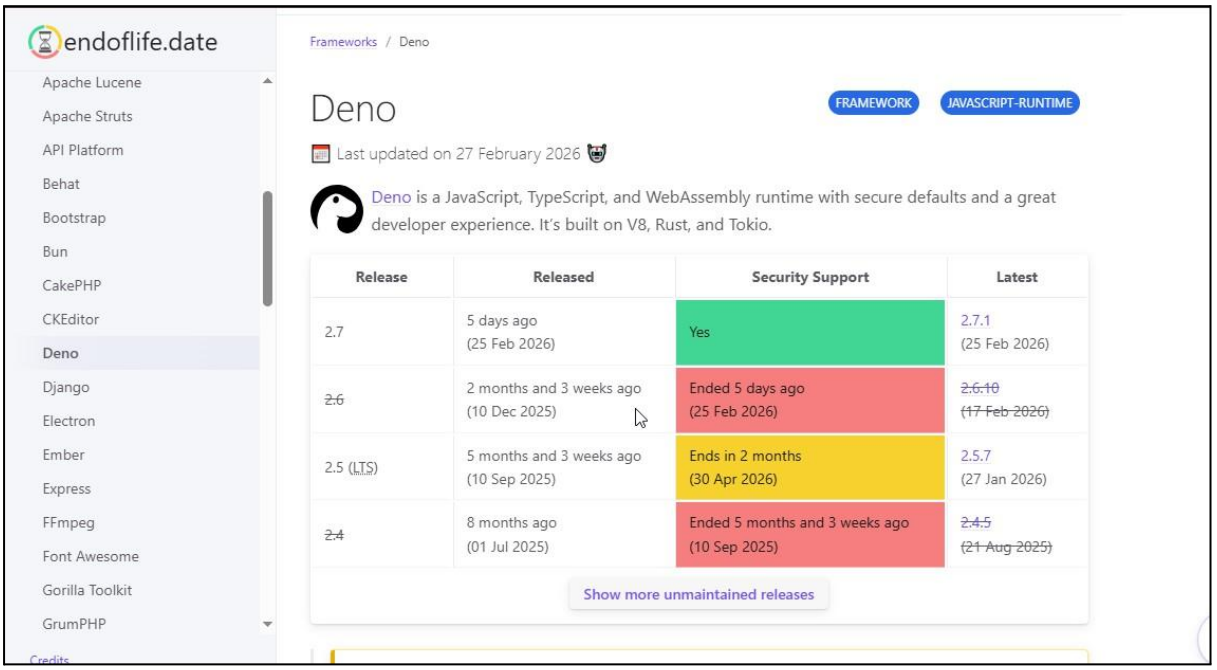
Criticality	Minor
Status	Resolved
CWE ID:	1104
CVSS3 Score:	4.0

Description

The source code review identified that the application is using an outdated version of the Deno JavaScript runtime, specifically version 2.6.4+, which has reached end of security support. Outdated runtime components may contain publicly disclosed vulnerabilities and security weaknesses that are addressed in later versions, potentially exposing the application environment to known security risks.
Using an unsupported or outdated runtime such as Deno 2.6.4 may expose the application to known vulnerabilities that could be exploited by attackers. These vulnerabilities may allow attackers to perform unauthorized actions, compromise application integrity, or affect the availability of services if exploited through runtime weaknesses or dependent components.

Proof of Concept: The below screenshot depicts that outdated deno 2.6.4 version is used which does not have any security support available.



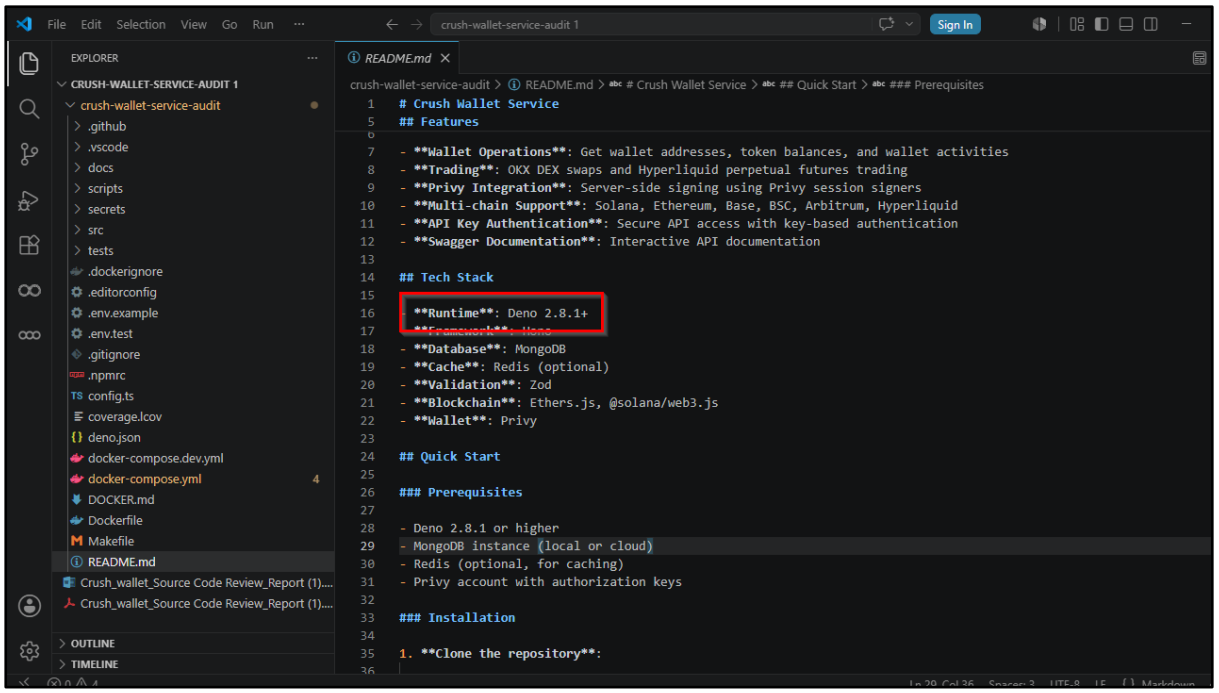


Recommendation

It is recommended to upgrade the runtime to a supported and actively maintained version of Deno, preferably the latest stable release. Keeping runtime environments updated ensures that known vulnerabilities are patched. Additionally, implement a dependency

monitoring process to regularly track security advisories and apply updates in a timely manner.

Revalidated Proof of Concept: The below screenshot depicts that deno 2.8.3 version is used which has security support available.



SE - Swagger Enabled

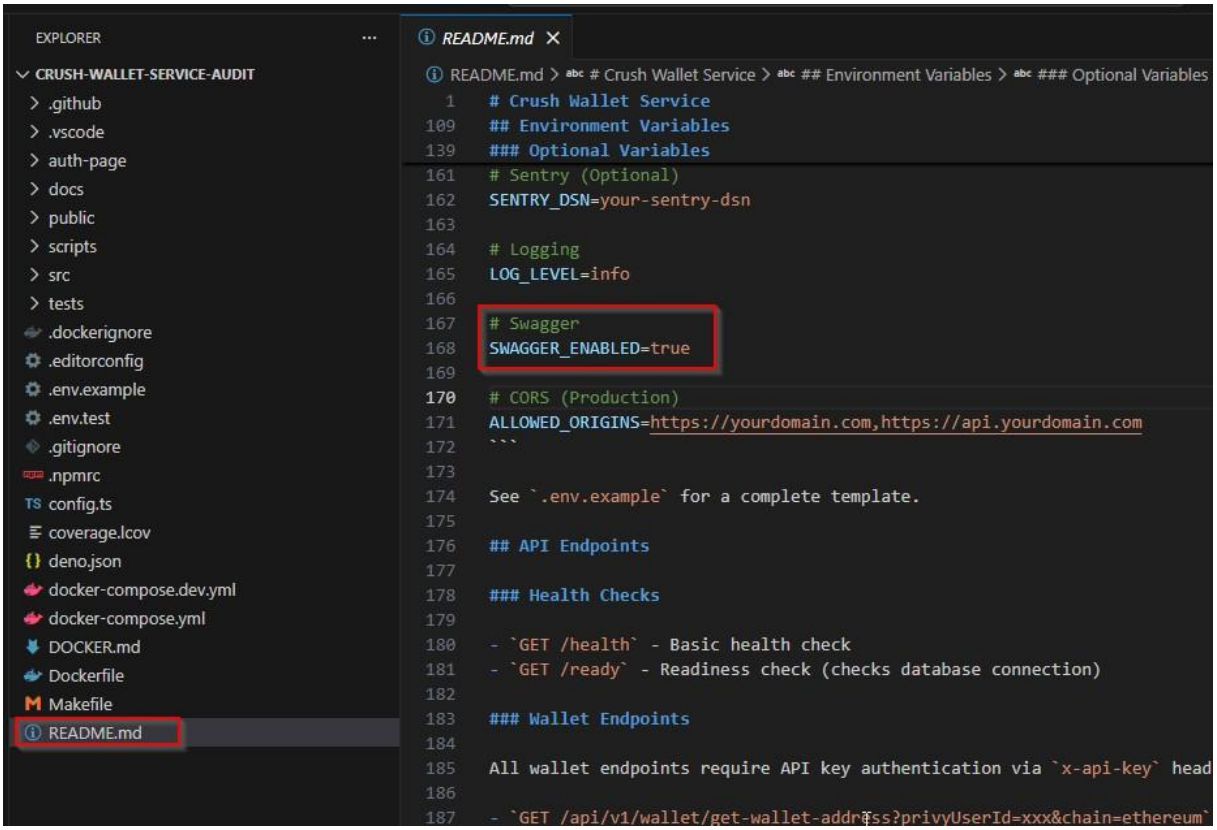
Criticality	Minor
Status	Resolved
CWE ID:	200
CVSS3 Score:	3.7

Description

The analysis identified that Swagger API documentation is enabled within the application environment. Swagger provides interactive API documentation that exposes endpoint details, request structures, parameters, and response formats, which may reveal internal API functionality and implementation details.

If exposed in production environments, Swagger documentation can assist attackers in enumerating available API endpoints and understanding application functionality. This information may facilitate targeted attacks, such as API abuse, injection attempts, or authentication bypass attempts, especially if security controls are not properly implemented.

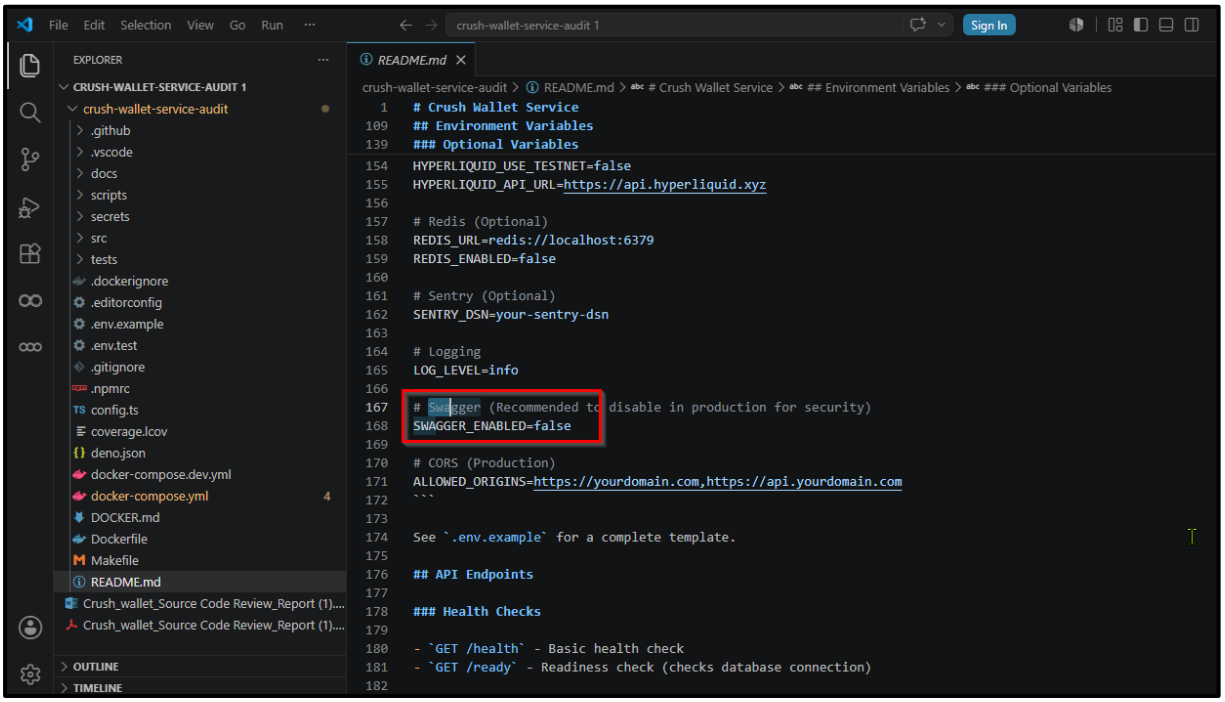
Proof of Concept: The below screenshot depicts that SWAGGER_ENABLED is set to True.



Recommendation

Swagger documentation should be disabled in production environments or restricted using authentication and network access controls. If documentation is required for operational purposes, it should only be accessible to authorized users within secure internal networks or protected development environments.

Revalidated Proof of Concept: The below screenshot depicts that SWAGGER_ENABLED is set to False.



ACAOHWW - Access-Control-Allow-Origin Header with Wildcard.

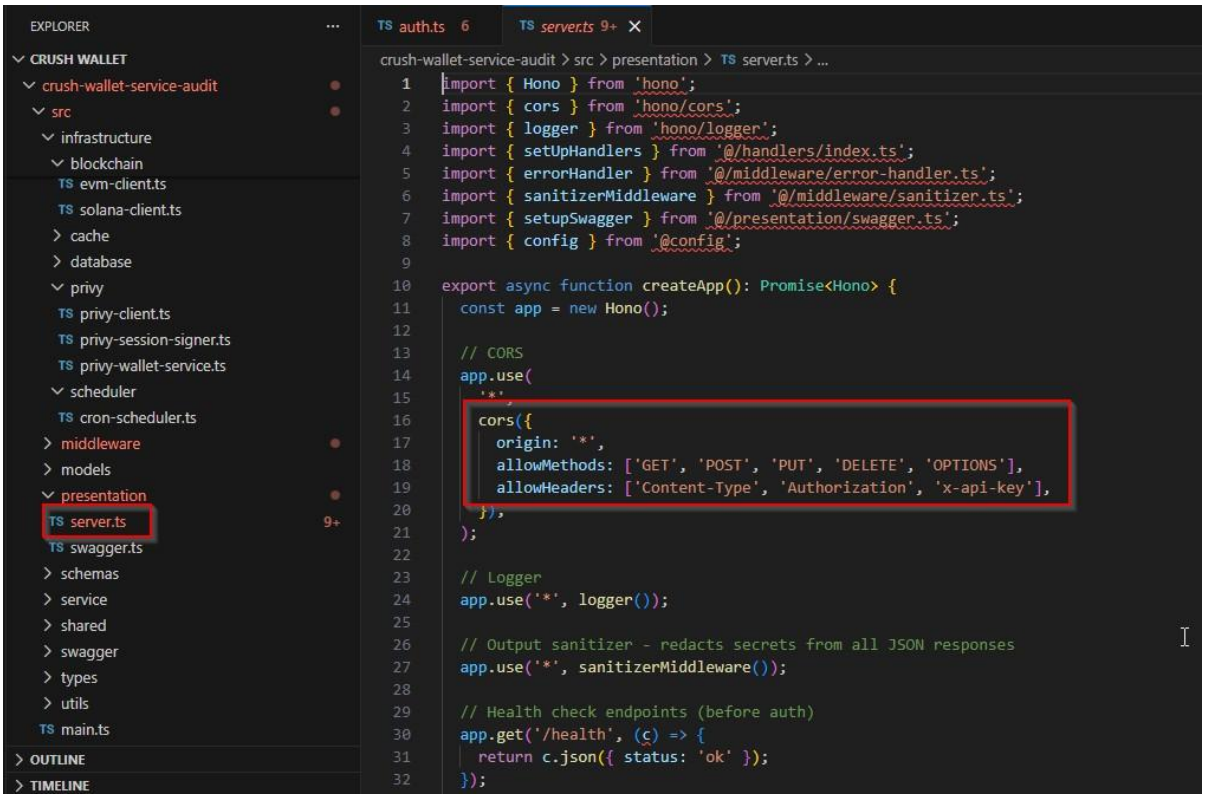
Criticality	Minor
Status	Resolved
CWE ID:	942
CVSS3 Score:	4.3

Description

The application response headers include the Access-Control-Allow-Origin header configured with a wildcard ("*"), allowing any external domain to make cross-origin requests to the application resources. This configuration permits unrestricted cross-domain interactions without validating trusted origins.

Permissive CORS configurations may allow malicious websites to interact with application resources through a victim's browser. If combined with authenticated sessions or sensitive APIs, attackers may exploit this configuration to perform unauthorized actions or retrieve sensitive data via cross-origin requests.

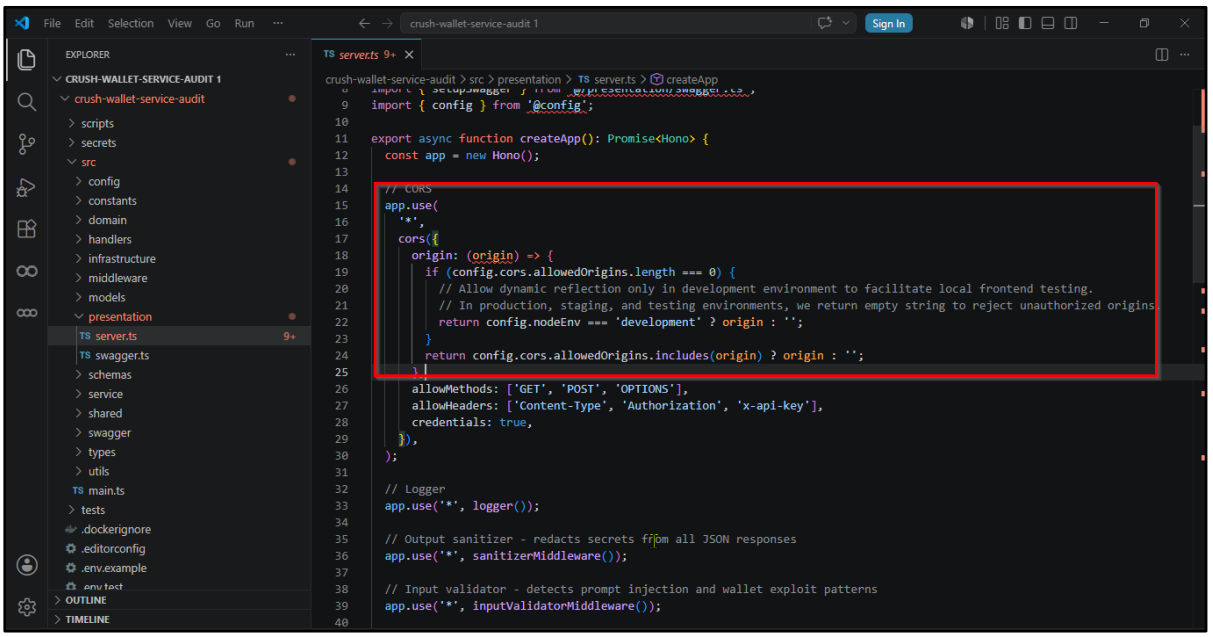
Proof of Concept: The below screenshot depicts that the Origin header is set to wildcard.



Recommendation

It is recommended to restrict the Access-Control-Allow-Origin header to specific trusted domains instead of using a wildcard. Only legitimate client domains that require access should be explicitly whitelisted. Additionally, implement proper authentication and validation controls to ensure secure cross-origin communication.

Revalidated Proof of Concept: The below screenshot depicts that the implementation has been updated to enforce an explicit origin allowlist via config.cors.allowedOrigins. Requests from unapproved origins are no longer allowed.



IME - Insecure Methods Enabled

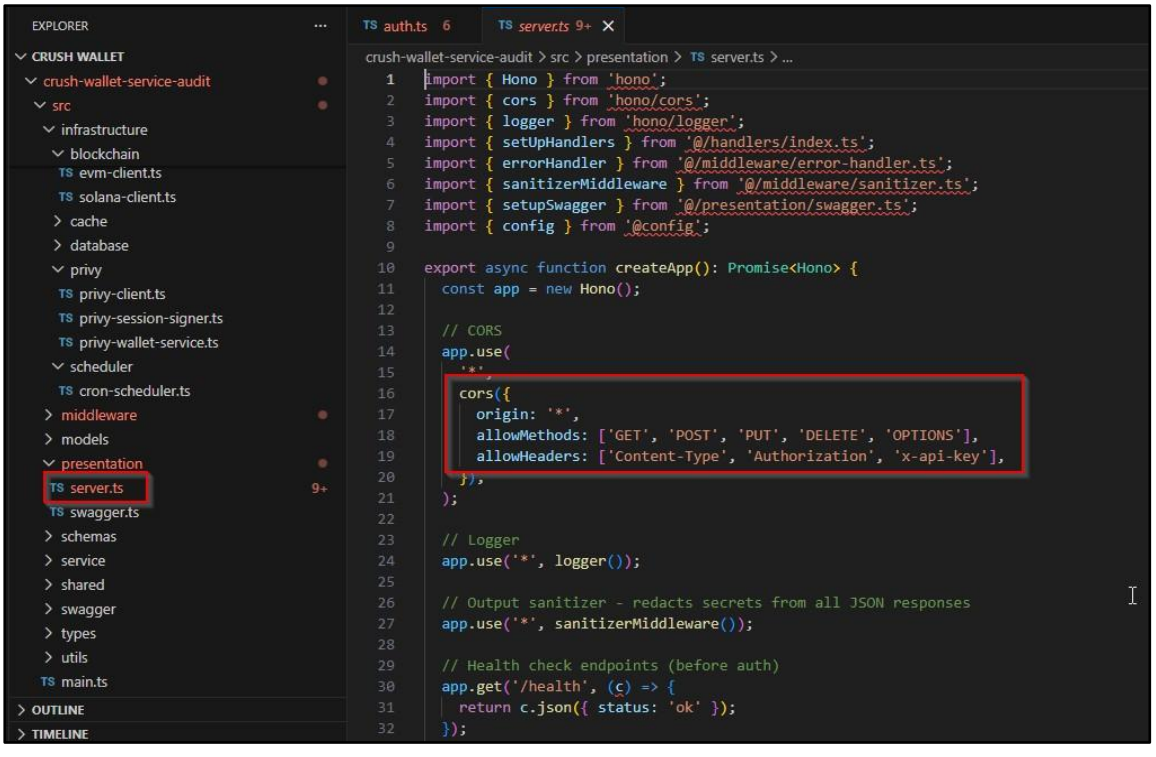
Criticality	Minor
Status	Resolved
CWE ID:	284
CVSS3 Score:	3.7

Description

The source code review identified that the application server allows additional HTTP methods such as PUT, DELETE, and OPTIONS. These methods are typically used for modifying or interacting with server resources and may not always be required for normal application functionality. Enabling unnecessary HTTP methods can increase the exposed attack surface of the application.

If these HTTP methods are not properly controlled or validated, attackers may attempt to exploit them to modify or delete application resources or perform unintended operations. Improper handling of PUT or DELETE requests may lead to unauthorized data manipulation, while the OPTIONS method can assist attackers in identifying supported methods during reconnaissance.

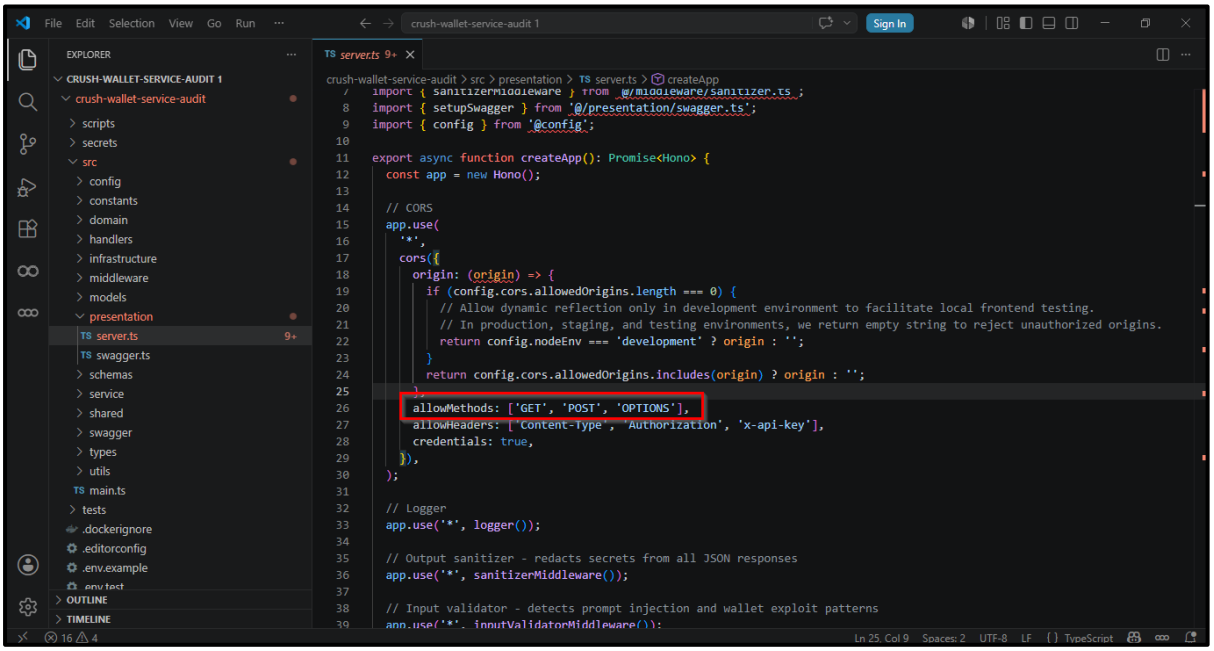
Proof of Concept: The below screenshot depicts that PUT, DELETE and OPTIONS methods are enabled.



Recommendation

It is recommended to restrict HTTP methods to only those strictly required for application functionality. If PUT, DELETE, or OPTIONS methods are not necessary, they should be disabled at the web server or application configuration level. Additionally, implement proper authentication and authorization controls for any endpoints that legitimately require these methods.

Revalidated Proof of Concept: The below screenshot depicts that GET, POST and OPTIONS methods are enabled.



Summary

The source code review identified several security weaknesses related to dependency management, configuration practices, authentication controls, and API exposure. The application was found to use an outdated runtime component, specifically Deno version 2.6.4, which has reached the end of security support and may contain known vulnerabilities. Additionally, secure secret management mechanisms such as Docker Secrets were not implemented, increasing the risk of credential exposure. Certain application functionalities were observed without proper authentication controls, potentially allowing unauthorized access. The configuration also permits permissive Cross-Origin Resource Sharing through a wildcard origin, enables additional HTTP methods including PUT, DELETE, and OPTIONS, and exposes API documentation via Swagger in the environment. Furthermore, container configuration issues such as base images not pinned by digest were observed, which may lead to inconsistent or unverified builds. Collectively, these weaknesses may increase the attack surface and should be addressed to improve the overall security posture of the application and align with secure coding and deployment best practices.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io